

CA267 Week 8 - Lab Materials – Instructions:

Beech Hill Library Project - Project Aims:

Beech Hill (primary and secondary) school has 800 students and a teaching staff of 50.

The library is open to the students all day during school hours and is currently operated by the English Department teaching staff on a rotation basis.

Providing an engaging library experience for students with diverse age, skill and competency levels is not easy. In installing a new library system it is envisaged that the students will be able to enjoy an intuitive, friendly and effective interaction with the wealth of information the library has to offer. This software solution must enable the library provide students with a positive and educational library experience.

The new library system will streamline library operations and improve the day-to-day efficiency of the staff. The school also wants to provide students, parents and educators with an affordable, easy-to-use, integrated software solution which will provide for effective management of the library. The solution will support the teachers and learners alike in making the most of the available learning resources - with access to information leading to learning and achievement.

The school require a computer based system that all teachers will be able to use and utilise in helping their students' access all library items. Records of books, multimedia materials and the like are tracked through the system to determine usage, availability and requests to reserve items.

The main functions of the system are outlined in the diagram below:

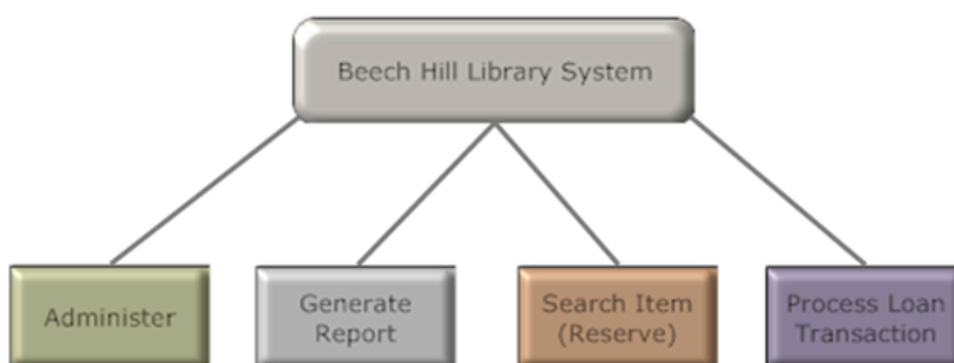


Figure 1- Beech Hill Library System Main Functions

Scenario:

Work in pairs - Work through the following tasks together.

Note: If the task doesn't specify the role that should work on the task then you should both work on it and should do so from the perspective of the test analyst.

Task 1.

One of you is working as a developer on the Beech Hill Library System and one of you is working as a Tester. As the developer, your manager has asked you to suggest three software development lifecycles that could be used for the development of the system. Work with the tester to provide a description of each of the three approaches and to explain to the manager how the approach taken to testing will differ depending on the approach that has been chosen. Which approach would you recommend and why?

Software development lifecycles that can be used:

1. Waterfall model

This model works on the principle that each phase of the software development (requirement analysis, system design, coding, testing and maintenance) is completed and well documented before moving on to the next one. There are no iterations, so when one phase is finished, we do not change the content after we move on. This would be one big project, and there would be only one final release. Since all of the phases are done only one time, the requirement analysis and system design are extensively discussed, planned and tested before coding begins. The actual testing of the code (including user acceptance testing) would be done only after the coding phase is finished, which introduces some risks since we don't know if the resulting software will satisfy the requirements stated at the beginning of the project.

2. Incremental model

This model allows for systems to be broken down into small components. We add on more and more functionality with each increment, starting with only partial functionality. This allows us to test the system against system and user acceptance requirements early enough so that it is not as expensive to make changes if any defects / inconsistencies are found. It will take multiple increments to get the full system functionality that was specified in the requirements specification.

3. Agile iterative model - e.g. Scrum

This model is similar to the incremental model in the sense of multiple iterations before we get the final functionality, and it requires a lot less planning. The focus of this type of model is delivering functionality piece by piece, and building upon previous iterations of the system. Since it is not typically well planned from the beginning, it can introduce some challenges when it is being tested against user requirements, but the testing occurs often, after every iteration. This would allow us to change our code if the requirements were wrongly interpreted or missing.

I would recommend the incremental approach, because it allows for more feedback, changes to be made early on, less cost in case of any defects and it breaks the project down into smaller, more manageable cycles that allow for best productivity of the development team.

Task 2.

One of you is working as a developer on the Beech Hill Library System and one of you is working as a Tester. Explain the types of skills and mind-set that would be ideal for each of the roles. As the tester, you have found a defect. Explain how you would communicate the defect to the developer. As the developer, explain how you might react. Explain the issues that may be encountered if a developer tests their own code or a tester runs their own tests. Suggest a better approach using the Beech Hill system as an example.

Answer: A tester should be naturally sceptical towards the system they are testing, but they should be considerate of the developer once they find a defect in the system. It is important to deliver the news in an appropriate, respectful manner, while still keeping their integrity as a tester. The developer should, however, also be receptive to constructive criticism and admit their mistake, not trying to prove the tester wrong. The interaction between members of the two roles should always be respectful and never condescending.

If a developer tests their own code, they might be biased, and interpret wrong outputs as right ones to confirm their bias. After the tester runs their own tests, these biases can be broken, and that is where problems start happening. A better approach would be to develop tests based on requirements of the system, in example of the Beech Hill system, a developer and a tester would work together at the start of the development lifecycle. For example, they need to develop and test a function that processes loan transactions. They would look at the exact specification of that function, for example, "after the loan is processed, go to the confirmation page showing the loan status, how long the loan will last etc." and make tests accordingly. A tester would be in charge of making test scripts, and since the requirement was clearly stated to the developer as well, they would know what to test and how to test it while debugging or doing component testing.

Task 3.

You are both developers working on the Beech Hill Library system. Your manager has advised that you will be using Pair Programming on this project. Explain what is meant by this approach. Do you agree with the Manager's decision to use Pair Programming for this project? Explain your rationale. As a team, discuss your expectation for how Pair programming will work on this project and the roles involved. What might go wrong and how might you address it? What might go right?

Answer: Pair Programming approach refers to programming in a pair where each person has a role. One person sits at a computer and programs, that person is called the driver, and the other person sits beside them and provides feedback on what the driver is doing, and they are called a navigator. The point of pair programming is to work on a solution together, and it can be done dynamically, so that both participants switch roles after a certain amount of time has passed. This is done to keep both participants engaged, because getting used to one specific role can become too repetitive for a participant and they might not deliver good results. It is important that the communication between two participants is constant, so that they know what the other person is thinking, and that way they can come to the solution quicker and effectively. Essentially, the driver has to program out loud, and the navigator is supposed to look at the bigger picture and ask questions and make suggestions to the driver.

I think that using Pair Programming can be a fun and effective way to work on a project, since it has been proven to be 15% more efficient, but it is also 15% more expensive. I would agree with the

Manager's decision only if it is within the budget that the school is willing to offer. Two people usually come up with a better solution in less time, but it still requires paying two people for one job. For this job, the roles would work as explained above. Two programmers would get together and work on one computer, implementing library functions together, with one person programming, and the other person making suggestions. They would, of course, switch their roles as a driver and a navigator every half an hour, to keep a dynamic environment.

What might go wrong: if there is a strain in a relationship between two programmers, working together might not be efficient at all. A strain could be caused either by personal disagreements, general disliking of one another, or incompatibility. Something simple like the lack of personal hygiene of one person could also steer another person away from being willing to cooperate. Two people could be too quiet, or not talk at all, or one person might be engaging but the other person is not, so there is freeloading happening. On the other hand, two people could like each other too much and become unprofessional in a work environment: chatting about unrelated things, being too loud, distracting other pairs etc.

What could go right: Two people with compatible mindsets and workflows can talk and come to an elegant solution for all functions, especially if one programmer's knowledge complements the other ones, so they fill each other's gaps and clarify any unambiguity. For example, one programmer knows how to deploy databases very well, so they might be a good navigator for the part of the programming where they work on a book database, and the other person might have good technical skills in backend implementation of basic library functions like loaning books. This would make the pair very efficient and the work would be done quickly and effectively.

Task 4.

You are both working as testers on the Beech Hill Library System. Your manager has read about low formality testing techniques and thinks that they shouldn't be used on the project. Explain to your manager how and when these techniques may be beneficial and work together to make a plan as to how they could be incorporated into the test plan for the Beech Hill Library system.

Answer: Low formality testing techniques may be beneficial because experienced testers have a good intuition about finding bugs, especially if they tested a similar program before. Since this is a library system, testers that have experience with large databases that support searching, loans, returns, etc. might know where to find the most common defects, or at least an area which is most error-prone. Exploratory testing is a fun and dynamic way to find defects, so testers will stay engaged and interested in finding them. This way of testing can help us to reveal some defects we would never expect. This type of testing is especially useful after the tester has found a defect and goes off script and tries to find more defects in the same pool, because we know they are more likely to occur in clusters. Based on exploratory testing, we can also evolve our planned scripted tests and improve on the process. I would agree with the manager that low formality testing should not be the only testing that is done for the entire system, but it would be a nice incorporation with scripted tests for reasons mentioned above. I would recommend that we do 30% exploratory testing, 70% scripting testing, as it is still important to have a well structured plan, but it is also good to leave a little bit of room for

testers to freely explore. This would definitely improve the experience and efficiency of testers, as they would be able to look for patterns and explore around bugs when they are found.

Task 5.

You are both working as developers on the Beech Hill Library system and have been tasked with completing the component testing of the system. Explain what takes place during this phase. Explain how stubs and drivers might be used during this phase of testing. Explain what is meant by integration strategy. Which integration strategy would you suggest if used for the Beech Hill Library system and why? Your manager thinks that some Alpha testing of the system should be planned. Do you agree with this decision and why?

Answer: Component testing is a phase in which we test a single unit or a collection of units, not the entire system. This is to make sure that components we developed are satisfying functional and non-functional requirements, for example, a component that tracks the time of a loan should functionally work, but it should also update quickly, which would be a non-functional requirement. This type of testing is usually done in the development phase, so not all components will be made by the time we need to test one. That is why we use stubs and drivers, which act as placeholders for components we are yet to implement. They simulate the real code, but they are not a fully functional implementation of it. They can be a separate program, a script that interacts with a code, or a 3rd party product. Stubs are specifically used in the top-down method, and they act as a component that is called by the component we are testing (for example, a loan mode might call for a timer component which tracks the amount of time the student has before they need to return the book, and the timer would be a stub), and driver is used in the bottom-up method, and it acts as a component that invokes the component that is being tested (the previous example would work the other way around here if we implement the timer first, and have a loan function as a driver).

Integration strategies are the ways we want to connect individual components together. They are usually connected by interfaces, and they can be connected from top down, bottom up or “big bang” - or in a non determined way. Top here refers to high-level modules, while bottom refers to low-level components. For the Beech Hill Library, I would recommend going top down, especially if the chosen development lifecycle is incremental, because this means that we add on functionality and details as we go. First we start with high-level modules, and then move on to more detailed, specific modules. I think this would be the most organised way to integrate and test all of the modules.

I agree with the decision that Alpha testing should be planned. This would give us a good insight into how users interact with the system and which parts of the system cause the most problems for them, since alpha testing seeks feedback from actual or potential users. Alpha testing is done at a supplier site, so good candidates for this testing would be school staff.

Task 6.

You are both working as testers for the Beech Hill Library System. Your manager is unsure of the difference between White Box and Black Box testing techniques. Explain the difference between the

two approaches. Which of the techniques do you think would be most effective for the Beech Hill Library System and why?

Answer: The White Box testing technique allows the tester to make tests based on the code of the system, while the Black Box testing technique allows the tester to make tests based only on requirements and specifications of the system. I think that the Black Box technique would be more effective in this case because it puts more importance on what the system is actually supposed to do for users like students of all ages, teachers of all ages etc. It gives us a better understanding of the user's point of view.

Task 7.

In the Beech Hill Library system, a student's profile includes their age. In order to ensure that the books are appropriate to the students reading level, books are put into categories based on age. The categories are as follows:

- Aged 4 – 8
- Aged 8 – 12
- Aged 12 – 18
- Aged 18 and over

- I. Using Equivalence Class Partitioning, explain what tests you would run based on testing these age categories.

Firstly I would identify valid and invalid partitions for each age category, for example in the first category, any age under 4 would be invalid, 4 to 8 inclusive would be valid, and any age greater than 8 would be invalid. I would write one test case for each invalid and valid partition, since all values in that partition are expected to produce the same output. I would also create a test partition that deals with only null values, empty strings, none etc, and then make a test case for each of these.

- II. Using Boundary Value Analysis, explain what test cases you would run based on testing these age categories.

I would run test cases that hold input values that are just above and just below the boundary value, and I would take three values for each boundary.

These are the test cases I would run:

- 3, 4, 5, 7, 8, 9
- 7, 8, 9, 11, 12, 13
- 11, 12, 13, 17, 18, 19
- 17, 18, 19

- III. Your manager thinks that only one of these techniques is needed. Are they correct in thinking this? Explain your rationale. If choosing one of the techniques, which would you choose and why?

The manager is not completely wrong in his thinking because boundary value analysis subsumes equivalence partitions for ranges, which means that equivalence partitioning is a part of boundary values. Once we determine boundary values, we know the partitions we can have for tests, but it is still useful to explicitly define all of the partitions. So if i had to choose only one of the techniques, I would choose boundary value analysis.

IV. What other tests might be needed here? Any other information you might want prior to testing this part of the system?

As mentioned prior, I would test for empty values, like a null value, an empty string, a 0 integer etc.

Since students' profiles contain their ages, I would test if appropriate books are shown to students' accounts based on their age, and I would test if age is correctly calculated based on the students' birthday.

I would also test extreme invalid input values, like negative numbers, very large numbers, etc.

Before testing, I would have to clarify whether the student that is 8 years old belongs to the 4 - 8 category or 8 - 12 category, since both categories are inclusive. I would need input validation rules, a mock database of students and requirements specification to draw tests from.